অধ্যায় ৭

০৭

Objects: Properties, Methods, Destructuring এবং Data Modeling

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Object Literal দিয়ে Object তৈরি করতে এবং সংক্ষিপ্ত লেখার রীতিগুলো ব্যবহার করতে।
- Dot ও Bracket Notation-এর পার্থক্য বুঝে সঠিকটি বেছে নিতে।
- Property পড়তে, যোগ করতে, বদলাতে, মুছতে এবং তার অস্তিত্ব নির্ভরযোগ্যভাবে যাচাই করতে।
- Method লিখতে এবং `this` কীভাবে কাজ করে তা ব্যাখ্যা করতে।
- Nested Object নিরাপদে পড়তে Optional Chaining ব্যবহার করতে।
- Destructuring দিয়ে Object ও Array থেকে মান বের করতে।
- `Object.keys()`, `Object.values()` ও `Object.entries()` দিয়ে Object-এর ওপর Loop চালাতে।
- Spread দিয়ে Shallow Copy তৈরি করতে এবং তার সীমা বুঝতে।
- JSON-এ রূপান্তর করে তথ্য সংরক্ষণ ও আদান-প্রদান করতে।
- বাস্তব তথ্যের জন্য একটি Object-এর গঠন (Data Model) নকশা করতে।
- একটি সম্পূর্ণ **Student Management System** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৬ সম্পন্ন করা: Array Method, Spread এবং Mutation বনাম নতুন মান।
- অধ্যায় ২-এর Reference: Object-এ ঠিকানা Copy হয়, মান নয়।
- অধ্যায় ৫-এর Function, Arrow Function এবং Scope।
- `chapter-07` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একজন শিক্ষার্থীর নাম, রোল, নম্বর, ফোন নম্বর এবং ঠিকানা আছে। এগুলো পাঁচটি আলাদা Variable-এ রাখা যায়, কিন্তু তখন কোন নম্বরটি কার, তা মনে রাখার দায় পুরোপুরি আপনার। একশ জন শিক্ষার্থী হলে পাঁচশ Variable। **Object** এই সমস্যার সমাধান: একটি জিনিসের সব তথ্য একটি নামে, প্রতিটি তথ্যের নিজস্ব লেবেলসহ।

আগের অধ্যায়গুলোতে আপনি Object ব্যবহার করেছেন, কিন্তু শুধু তার সবচেয়ে সরল রূপে। বাস্তব Program-এর তথ্য প্রায় সব সময় Object আকারে আসে: Server থেকে আসা উত্তর, Form-এর ডেটা, Database-এর একটি সারি, Settings-এর তালিকা। Object ভালোভাবে না জানলে এসব তথ্য নিয়ে কাজ করা যায় না।

এই অধ্যায়ের শেষ ভাগে একটি গুরুত্বপূর্ণ দক্ষতা আসবে: তথ্যকে কীভাবে সাজাবেন, তা আগে থেকে ঠিক করা। ভালো Data Model থাকলে পরের সব Code সহজ হয়; দুর্বল Model থাকলে প্রতিটি Function-এ বাড়তি যাচাই লিখতে হয়।

৭.১ Object কেন প্রয়োজন

Array তথ্য রাখে ক্রম অনুযায়ী: প্রথম, দ্বিতীয়, তৃতীয়। Object তথ্য রাখে নাম অনুযায়ী।

why-objects.js · JavaScript — চলবে: Node.js

```
const student = {
  name: 'Sara',
  age: 20,
  marks: 88,
};

console.log(student.name);
console.log(student.marks);
console.log(student);
```

Output

```
Sara
88
{ name: 'Sara', age: 20, marks: 88 }
```

Object-এর প্রতিটি নাম-মানের জোড়াকে **Property** বলে। নামটিকে **Key**, আর তার সঙ্গে যুক্ত মানটিকে **Value**। `name`, `age` ও `marks` এখানে তিনটি Key।

	Array	Object
তথ্য খোঁজা হয়	অবস্থান (Index) দিয়ে	নাম (Key) দিয়ে
উপযুক্ত	একই ধরনের অনেক জিনিসের তালিকা	একটি জিনিসের বিভিন্ন বৈশিষ্ট্য
উদাহরণ	সব শিক্ষার্থী	একজন শিক্ষার্থী

বাস্তবে দুটো একসঙ্গে ব্যবহৃত হয়: শিক্ষার্থীদের একটি Array, যার প্রতিটি Element একটি Object। অধ্যায় ৪ ও ৬-এ আপনি এই কাঠামোই ব্যবহার করেছেন।

> **নোট:** Object সরাসরি `console.log()` করলে Node.js ও Browser ভিন্ন সাজে দেখায়। এই অধ্যায়ে পুরো Object দেখানো উদাহরণগুলো Node.js-এর Output; Browser Console একই তথ্য দেখায়, তবে আলাদা চেহারায়।

৭.২ Object তৈরি করা

সবচেয়ে প্রচলিত উপায় **Object Literal**: দ্বিতীয় বন্ধনীর ভেতরে কমা দিয়ে আলাদা করা `key: value` জোড়া। শেষ Property-এর পরের কমাটি ঐচ্ছিক, কিন্তু রাখলে নতুন Property যোগ করার সময় ভুল কম হয়।

দুটি সংক্ষিপ্ত রীতি

shorthand.js · JavaScript — চলবে: Node.js

```
const name = 'Rafi';
const marks = 70;

const student = { name, marks };
console.log(student);

const field = 'city';
const place = { [field]: 'Dhaka' };
console.log(place);
```

Output

```
{ name: 'Rafi', marks: 70 }
{ city: 'Dhaka' }
```

- **Shorthand Property** — Key এবং Variable-এর নাম এক হলে শুধু একবার লিখলেই চলে। `{ name, marks }` মানে `{ name: name, marks: marks }`।
- **Computed Key** — বর্গবন্ধনীর ভেতরে একটি Expression দিলে তার মান Key হয়ে যায়। Key আগে থেকে জানা না থাকলে এটি কাজে লাগে।

Object তৈরির আরও কিছু পদ্ধতি আছে, যেমন `new Object()` এবং `Object.create()`। পুরোনো Code-এ এগুলো দেখা যায়, কিন্তু সাধারণ তথ্য রাখার জন্য Literal-ই সবচেয়ে পরিষ্কার। `Object.create()`-এর আসল ব্যবহার Prototype-এর সঙ্গে জড়িত, যা অধ্যায় ১৩-তে আসবে।

৭.৩ Dot এবং Bracket Notation

Property পড়ার দুটি উপায়: ডট দিয়ে (`profile.name`), অথবা বর্গবন্ধনীতে Key-কে String হিসেবে দিয়ে (`profile['name']`)।

dot-bracket.js · JavaScript — চলবে: Browser Console ও Node.js

```
const profile = { name: 'Mina', 'home-city': 'Khulna' };

console.log(profile.name);
console.log(profile['name']);
console.log(profile['home-city']);

const key = 'name';
console.log(profile[key]);
console.log(profile.key);
```

Output

```
Mina
Mina
Khulna
Mina
undefined
```

পরিস্থিতি	ব্যবহার করুন
Key আগে থেকে জানা এবং সাধারণ নাম	Dot: `profile.name`
Key-তে হাইফেন, ফাঁকা জায়গা বা সংখ্যা দিয়ে শুরু	Bracket: `profile['home-city']`
Key একটি Variable-এ আছে	Bracket: `profile[key]`

শেষ লাইনটি সবচেয়ে প্রচলিত ভুলটি দেখায়। `profile.key` লিখলে JavaScript `key` নামের Variable-এর মান ব্যবহার করে না; সে `'key'` নামের একটি Property খোঁজে, যা নেই। Variable-এর মান দিয়ে খুঁজতে চাইলে সব সময় বর্গবন্ধনী।

৭.৪ Property পড়া, যোগ, বদল এবং মোছা

Object তৈরির পরেও তাতে Property যোগ, বদল বা মোছা যায়।

property-changes.js · JavaScript চলবে: Node.js

```
const book = { title: 'JavaScript', price: 500 };

book.author = 'Moynul';
book.price = 450;
console.log(book);

delete book.author;
console.log(book);
console.log(book.author);
```

Output

```
{ title: 'JavaScript', price: 450, author: 'Moynul' }
{ title: 'JavaScript', price: 450 }
undefined
```

- নেই এমন Key-তে মান রাখলে নতুন Property যোগ হয়; আছে এমন Key-তে রাখলে মান বদলায়। দুটো দেখতে একই রকম।
- `delete` একটি Property সম্পূর্ণ সরিয়ে দেয়।
- নেই এমন Property পড়লে Error হয় না; `undefined` পাওয়া যায়।
- `book` একটি `const`, তবু ভেতরের তথ্য বদলানো যাচ্ছে (অধ্যায় ২): `const` শুধু নামটিকে একই Object-এর সঙ্গে বেঁধে রাখে।

Property আছে কি না

`book.author === undefined` দিয়ে যাচাই নির্ভরযোগ্য নয়, কারণ একটি Property থাকতে পারে, আর তার মানই হতে পারে `undefined`।

has-property.js · JavaScript চলবে: Browser Console ও Node.js

```
const book = { title: 'JavaScript', discount: undefined };

console.log(book.discount === undefined);
console.log(book.author === undefined);

console.log(Object.hasOwn(book, 'discount'));
console.log(Object.hasOwn(book, 'author'));

console.log('toString' in book);
console.log(Object.hasOwn(book, 'toString'));
```

Output

```
true
true
true
false
true
false
```

`=== undefined` দুটি ক্ষেত্রকে আলাদা করতে পারেনি। `Object.hasOwn(object, key)` পেরেছে: `discount` আছে (মান `undefined` হলেও), `author` নেই।

`in` Operator-ও অস্তিত্ব যাচাই করে, কিন্তু সে Object-এর নিজের Property ছাড়াও উত্তরাধিকারসূত্রে পাওয়া Property খোঁজে। প্রতিটি Object জন্ম থেকেই `toString`-এর মতো কিছু Method পায়, তাই `'toString' in book` সত্য। নিজের তথ্য যাচাই করতে `Object.hasOwn()` ব্যবহার করুন।

৭.৫ Method এবং this

Property-র মান একটি Function-ও হতে পারে। এমন Property-কে **Method** বলে। `console.log`, `array.push`, `text.trim` —এগুলো সবই Method।

methods.js · JavaScript — চলবে: Browser Console ও Node.js

```
const account = {
  owner: 'Sara',
  balance: 1000,

  deposit(amount) {
    this.balance += amount;
    return this.balance;
  },

  describe() {
    return `${this.owner}: ${this.balance} টাকা`;
  },
};

console.log(account.deposit(500));
console.log(account.describe());
```

Output

```
1500
Sara: 1500 টাকা
```

`deposit(amount) { ... }` হলো Method লেখার সংক্ষিপ্ত রূপ; এর অর্থ `deposit: function (amount) { ... }`।

Method-এর ভেতরে **`this`** সেই Object-কে বোঝায়, যার মাধ্যমে Method-টি Call করা হয়েছে। `account.deposit(500)`-এ ডটের বাঁ পাশে `account`, তাই `this` হলো `account`, এবং `this.balance` মানে `account.balance`।

this হারিয়ে যাওয়া

`this` ঠিক হয় Method কীভাবে Call করা হলো তা থেকে, কোথায় লেখা হয়েছে তা থেকে নয়। Method-টিকে Object থেকে আলাদা করে Call করলে `this` আর Object-টি থাকে না।

lost-this.js · JavaScript — চলবে: Browser Console ও Node.js

```
'use strict';

const account = {
  owner: 'Sara',
  describe() {
    return `${this.owner}`;
  },
};

const describe = account.describe;
console.log(describe());
```

Error Message

```
TypeError: Cannot read properties of undefined (reading 'owner')
```

`describe()`-এর বাঁ পাশে কোনো Object নেই, তাই Strict Mode-এ `this` হয় `undefined`। Function-টি একই, শুধু Call করার ধরন বদলেছে। এই সমস্যা সবচেয়ে বেশি দেখা দেয় যখন একটি Method-কে Callback হিসেবে পাঠানো হয়।

> **সতর্কতা:** Object-এর Method Arrow Function দিয়ে লিখবেন না। Arrow Function-এর নিজস্ব `this` থাকে না; সে বাইরের Scope-এর `this` ব্যবহার করে, যা Object-টি নয়। `describe: () => this.owner` লিখলে `undefined` পাওয়া যায়। Method-এর জন্য ওপরের সংক্ষিপ্ত রূপটি ব্যবহার করুন।

`this` এবং Object-এর আচরণ নিয়ে পূর্ণ আলোচনা Class-এর সঙ্গে অধ্যায় ১৩-তে।

৭.৬ Nested Object এবং Optional Chaining

একটি Property-র মান আরেকটি Object হতে পারে। এভাবে তথ্যকে স্তরে স্তরে সাজানো যায়।

nested.js · JavaScript চলবে: Browser Console ও Node.js

```
const student = {
  name: 'Rafi',
  contact: {
    phone: '01700-000002',
    address: { city: 'Bogura' },
  },
};

const newcomer = { name: 'Nila' };

console.log(student.contact.address.city);
console.log(newcomer.contact?.address?.city);
console.log(newcomer.contact?.address?.city ?? 'অজানা');
```

Output

```
Bogura
undefined
অজানা
```

`student.contact.address.city` প্রতিটি ধাপে এক স্তর ভেতরে যায়। কিন্তু `newcomer`-এর `contact` নেই। সেখানে সাধারণ ডট ব্যবহার করলে কী হয়:

missing-nested.js · JavaScript চলবে: Browser Console ও Node.js

```
const newcomer = { name: 'Nila' };

console.log(newcomer.contact.address);
```

Error Message

```
TypeError: Cannot read properties of undefined (reading 'address')
```

`newcomer.contact` হলো `undefined`, আর `undefined`-এর কোনো Property পড়া যায় না। **Optional Chaining** (`?.`) এই সমস্যার সমাধান: বাঁ পাশের মান `null` বা `undefined` হলে সে আর এগোয় না, Error-এর বদলে `undefined` দেয়। তার সঙ্গে অধ্যায় ৩-এর `??` জুড়ে দিলে একটি পূর্বনির্ধারিত মান দেখানো যায়।

`?.` ব্যবহার করুন শুধু সেখানে, যেখানে তথ্যটি সত্যিই না-ও থাকতে পারে, যেমন ঐচ্ছিক যোগাযোগের তথ্য। প্রতিটি ডটের জায়গায় `?.` বসালে আসল Bug লুকিয়ে যায়: যে তথ্য অবশ্যই থাকার কথা, সেটি না থাকলে Error দেখাই ভালো।

৭.৭ Destructuring

Object থেকে কয়েকটি Property বের করে আলাদা Variable-এ রাখার সংক্ষিপ্ত উপায় **Destructuring**।

destructuring.js · JavaScript চলবে: Browser Console ও Node.js

```
const student = { id: 'S1', name: 'Sara', marks: 88 };

const { name, marks } = student;
console.log(name, marks);

const { marks: score, grade = 'জানা নেই' } = student;
console.log(score, grade);

function describe({ name, marks }) {
  return `${name} পেয়েছেন ${marks}`;
}

console.log(describe(student));
```

Output

```
Sara 88
88 জানা নেই
Sara পেয়েছেন 88
```

- `const { name, marks } = student;` — `student.name` যায় `name`-এ, `student.marks` যায় `marks`-এ। নাম মিলতে হয়; ক্রম গুরুত্বপূর্ণ নয়।
- `marks: score` — Property-টি অন্য নামে নেওয়া। এখানে কোলনের বাঁ পাশে Property-র নাম, ডান পাশে নতুন Variable-এর নাম।
- `grade = 'জানা নেই'` — Default মান। Property না থাকলে বা `undefined` হলে এটি বসে; অধ্যায় ৫-এর Default Parameter-এর মতোই, `null` হলে বসে না।
- **Parameter-এ Destructuring** — Function একটি Object পায় এবং সঙ্গে সঙ্গে তার দরকারি Property-গুলো আলাদা করে নেয়। কোন তথ্য দরকার, তা Function-এর প্রথম লাইনেই স্পষ্ট হয়।

Array-কেও Destructure করা যায়, তবে সেখানে নাম নয়, অবস্থান মেলে:

array-destructuring.js · JavaScript চলবে: Browser Console ও Node.js

```
const [first, second] = ['আম', 'কলা', 'লিচু'];

console.log(first, second);
```

Output

```
আম কলা
```

৭.৮ Object-এর ওপর Loop

Object-এর সব Property নিয়ে কাজ করার জন্য তিনটি Method আছে। প্রতিটি একটি Array ফেরত দেয়, তাই অধ্যায় ৬-এর সব Array Method তার ওপর ব্যবহার করা যায়।

object-loops.js · JavaScript — চলবে: Node.js

```
const marks = { bangla: 80, english: 72, math: 91 };

console.log(Object.keys(marks));
console.log(Object.values(marks));

for (const [subject, mark] of Object.entries(marks)) {
  console.log(`${subject}: ${mark}`);
}

const total = Object.values(marks).reduce((sum, mark) => sum + mark, 0);
console.log(total);
```

Output

```
[ 'bangla', 'english', 'math' ]
[ 80, 72, 91 ]
bangla: 80
english: 72
math: 91
243
```

Method	ফেরত দেয়
`Object.keys(obj)`	সব Key-এর Array
`Object.values(obj)`	সব Value-এর Array
`Object.entries(obj)`	`[key, value]` জোড়ার Array

`Object.entries()`-এর প্রতিটি Element একটি দুই-Element-এর Array। `for (const [subject, mark] of ...)` লেখায় অনুচ্ছেদ ৭.৭-এর Array Destructuring প্রতিটি জোড়াকে দুটি নামে ভাগ করে নিচ্ছে।

> **নোট:** `for...in` নামে একটি পুরোনো Loop-ও Object-এর Key দেয়, কিন্তু সেটি উত্তরাধিকারসূত্রে পাওয়া Property-ও দেখাতে পারে। নিজের তথ্যের জন্য `Object.keys()` বা `Object.entries()` ব্যবহার করুন।

৭.৯ Spread, Rest এবং Shallow Copy

অধ্যায় ৬-এ Array-র জন্য যে Spread (`...`) দেখেছেন, Object-এও তা কাজ করে।

spread-copy.js · JavaScript — চলবে: Node.js

```
const original = { name: 'Mina', marks: 74, contact: { city: 'Khulna' } };

const updated = { ...original, marks: 80 };
console.log(original.marks, updated.marks);

updated.contact.city = 'Dhaka';
console.log(original.contact.city);

const { contact, ...withoutContact } = original;
console.log(withoutContact);
```

Output

```
74 80
Dhaka
{ name: 'Mina', marks: 74 }
```

- `{ ...original, marks: 80 }` — `original`-এর সব Property নতুন Object-এ, তারপর `marks` নতুন মানে। একই Key দুবার থাকলে পরেরটি জেতে। অধ্যায় ৬-এর Inventory Project-এ মজুত বদলাতে ঠিক এটিই ব্যবহার করা হয়েছিল।
- **কিন্তু Copy-টি Shallow!** প্রথম স্তরের Property-গুলো নতুন Object-এ গেছে, কিন্তু `contact` একটি Object, আর তার জন্য শুধু ঠিকানা Copy হয়েছে। তাই `updated.contact.city` বদলালে `original.contact.city`-ও বদলে গেছে।
- **Rest** — Destructuring-এ `...withoutContact` "বাকি সব" Property একটি নতুন Object-এ সংগ্রহ করে। একটি Property বাদ দিয়ে Copy তৈরির পরিষ্কার উপায় এটি।

সম্পূর্ণ আলাদা Copy

ভেতরের স্তরসহ পুরোপুরি আলাদা Copy দরকার হলে `structuredClone()` ব্যবহার করুন। এটি আধুনিক Browser ও Node.js-এ পাওয়া যায়।

deep-copy.js · JavaScript — চলবে: Browser Console ও Node.js

```
const original = { name: 'Mina', contact: { city: 'Khulna' } };
const deep = structuredClone(original);

deep.contact.city = 'Sylhet';

console.log(original.contact.city);
console.log(deep.contact.city);
```

Output

```
Khulna
Sylhet
```

ভেতরের একটি তথ্য বদলাতে পুরো Object-এর Deep Copy সাধারণত লাগে না। বরং যে স্তর বদলাচ্ছে, শুধু সেই স্তরে আবার Spread করুন: `{ ...student, contact: { ...student.contact, city: 'Sylhet' } }`।

৭.১০ Reference, সমতা এবং Freeze

অধ্যায় ২ থেকে: Object Variable-এ ঠিকানা রাখে, আর `===` ঠিকানা তুলনা করে।

equality.js · JavaScript — চলবে: Browser Console ও Node.js

```
const a = { id: 1 };
const b = { id: 1 };
const c = a;

console.log(a === b);
console.log(a === c);
console.log(JSON.stringify(a) === JSON.stringify(b));
```

Output

```
false
true
true
```

একই বিষয়বস্তুর দুটি আলাদা Object `===` দিয়ে সমান নয়। বিষয়বস্তু তুলনা করতে Property ধরে ধরে মেলাতে হয়। ছোট, সরল Object-এর জন্য দুটিকেই JSON String-এ রূপান্তর করে তুলনা করা যায় (অনুচ্ছেদ ৭.১১), তবে Property-র ক্রম আলাদা হলে এই পদ্ধতি ভুল উত্তর দেয়।

Object.freeze

একটি Object যেন আর বদলানো না যায়, তা নিশ্চিত করতে `Object.freeze()` ব্যবহার করা হয়।

freeze.js · JavaScript — চলবে: Browser Console ও Node.js

```
const settings = Object.freeze({ theme: 'light', size: { font: 14 } });

settings.theme = 'dark';
console.log(settings.theme);

settings.size.font = 18;
console.log(settings.size.font);
```

Output

```
light
18
```

`theme` বদলায়নি। কিন্তু `freeze` -ও Shallow: ভেতরের `size` Object জমাট নয়, তাই সেটি বদলে গেছে। আরেকটি বিষয়: এই File-এ Strict Mode নেই, তাই `settings.theme = 'dark'` নীরবে উপেক্ষিত হয়েছে। `'use strict';` থাকলে একই লাইন `TypeError` দিত, যা ভুল ধরার জন্য ভালো।

৭.১১ JSON: তথ্য সংরক্ষণ ও আদান-প্রদান

JSON (JavaScript Object Notation) হলো তথ্যকে লেখা (String) হিসেবে প্রকাশের একটি মান-নির্ধারিত রূপ। দেখতে JavaScript Object-এর মতো, কিন্তু এটি শুধু একটি String। Server-এর সঙ্গে তথ্য আদান-প্রদান, File-এ সংরক্ষণ এবং Browser Storage—সব জায়গায় JSON ব্যবহৃত হয়।

json.js · JavaScript চলবে: Browser Console ও Node.js

```javascript
const student = { name: 'Sara', marks: 88, passed: true, contact: null };

const text = JSON.stringify(student);
console.log(text);
console.log(typeof text);

const back = JSON.parse(text);
console.log(back.name, back.marks);
console.log(back === student);
```

Output

```
{"name":"Sara","marks":88,"passed":true,"contact":null}
string
Sara 88
false
```

- **`JSON.stringify(value)`** — Object থেকে JSON String।
- **`JSON.parse(text)`** — JSON String থেকে নতুন Object। ফেরত আসা Object মূলটির একটি সম্পূর্ণ আলাদা Copy।

JSON-এর নিয়ম JavaScript-এর চেয়ে কঠোর: Key সব সময় Double Quote-এ, String-ও Double Quote-এ, শেষে বাড়তি কমা চলে না। সব ধরনের মান JSON-এ রাখাও যায় না।

json-limits.js · JavaScript চলবে: Browser Console ও Node.js

```
const record = {
  name: 'Rafi',
  note: undefined,
  greet() {
    return 'hi';
  },
  joined: new Date(0),
};

console.log(JSON.stringify(record));
console.log(JSON.stringify({ name: 'Rafi', marks: 70 }, null, 2));
```

Output

```
{"name":"Rafi","joined":"1970-01-01T00:00:00.000Z"}
{
  "name": "Rafi",
  "marks": 70
}
```

`undefined` মানের Property এবং Method বাদ পড়েছে, আর তারিখ একটি String হয়ে গেছে; `JSON.parse()` সেটিকে আবার তারিখে ফেরাবে না। `JSON.stringify()` -এর তৃতীয় Argument (`2`) Output-কে সাজিয়ে দেখায়, যা পড়ার জন্য সুবিধাজনক।

ভুল JSON পার্স করতে গেলে Program থেমে যায়:

bad-json.js · JavaScript চলবে: Browser Console ও Node.js

```
const text = "{name: 'Sara'}";

console.log(JSON.parse(text));
```

Error Message

```
SyntaxError: Expected property name or '}' in JSON at position 1
```

বাইরে থেকে আসা JSON সব সময় ঠিক থাকবে, এমন ধরে নেওয়া যায় না। এমন Error ধরে সামলানোর উপায় অধ্যায় ১১-এ শেখানো হবে।

Browser-এ সংরক্ষণ: localStorage

Browser-এ ছোট তথ্য স্থায়ীভাবে রাখার একটি সহজ উপায় `localStorage`। এটি শুধু String রাখে, তাই Object রাখতে JSON-এ রূপান্তর লাগে। Node.js-এ এটি নেই; উদাহরণটি Browser-এ চালান।

storage.js · JavaScript চলবে: Browser

```
const student = { name: 'Sara', marks: 88 };

localStorage.setItem('student', JSON.stringify(student));

const saved = JSON.parse(localStorage.getItem('student'));
console.log(saved.name, saved.marks);
console.log(localStorage.getItem('missing'));

localStorage.removeItem('student');
```

Output

```
Sara 88
null
```

Page বন্ধ করে আবার খুললেও `localStorage`-এর তথ্য থাকে। নেই এমন Key পড়লে `null` আসে। Password বা অন্য গোপন তথ্য এখানে রাখবেন না: একই Website-এর যেকোনো Script এটি পড়তে পারে।

৭.১২ Data Modeling: তথ্যের গঠন নকশা করা

Code লেখার আগে ঠিক করুন একটি জিনিসের তথ্য কোন আকারে থাকবে। এই নকশাকে **Data Model** বা Schema বলে। Project-এর একজন শিক্ষার্থীর Model:

Property	Type	নিয়ম	উদাহরণ
`id`	String	আবশ্যক; ফাঁকা নয়; প্রত্যেকের আলাদা	`'S1'`
`name`	String	আবশ্যক; কমপক্ষে ২ অক্ষর	`'Sara Ahmed'`
`marks`	Number	০ থেকে ১০০-এর মধ্যে পূর্ণসংখ্যা	`88`
`contact`	Object বা `null`	ঐচ্ছিক	`{ phone: '01700-000001' }`

এই সারণি থেকে কয়েকটি সিদ্ধান্ত সরাসরি আসে:

- **একটি অনন্য পরিচয়।** দুজন শিক্ষার্থীর নাম একই হতে পারে, তাই খোঁজা, বদলানো ও মোছার জন্য `id` লাগে।
- **একটি জিনিস, এক জায়গায়।** Grade সংরক্ষণ করা হয় না, কারণ সেটি নম্বর থেকে হিসাব করা যায়। দুটোই রাখলে একটি বদলালে অন্যটি পুরোনো থেকে যেতে পারে।
- **"নেই" বোঝাতে `null`।** যোগাযোগের তথ্য না থাকলে `contact: null`, যাতে প্রতিটি শিক্ষার্থীর Object-এ একই Property থাকে।

- **ঐচ্ছিক তথ্য আলাদা স্তরে।** ফোন ও ঠিকানা `contact`-এর ভেতরে, যাতে পড়ার সময় `contact?.phone` দিয়ে নিরাপদে পাওয়া যায়।

৭.১৩ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
Variable-এর মান দিয়ে খুঁজতে `obj.key` লেখা	`undefined`	`obj[key]`
Spread Copy-কে সম্পূর্ণ আলাদা ভাবা	ভেতরের Object বদলালে মূলটিও বদলায়	যে স্তর বদলাবেন সেখানে আবার Spread, অথবা `structuredClone()`
`=== undefined` দিয়ে অস্তিত্ব যাচাই	মান `undefined` হলে ভুল উত্তর	`Object.hasOwn()`
Method Arrow Function দিয়ে লেখা	`this` থেকে `undefined`	Method-এর সংক্ষিপ্ত রূপ
Method আলাদা করে Callback হিসেবে পাঠানো	`TypeError: Cannot read properties of undefined`	`() => account.describe()` পাঠান
প্রতিটি ডটের জায়গায় `?.`	আসল Bug চাপা পড়ে	শুধু ঐচ্ছিক তথ্যে `?.`
দুটি Object `===` দিয়ে তুলনা	একই বিষয়বস্তুতেও `false`	Property ধরে তুলনা করুন
`Object.freeze()`-কে সম্পূর্ণ সুরক্ষা ভাবা	ভেতরের Object বদলায়	`freeze` Shallow
JSON-এ Single Quote বা শেষে কমা	`SyntaxError`	Double Quote; শেষে কমা নয়
JSON-এ তারিখ রাখা	ফেরত আসে String হয়ে	পড়ার পর `new Date(text)` দিয়ে রূপান্তর
Argument-এর Object বদলানো	Call-কারীর তথ্য বদলে যায়	নতুন Object ফেরত দিন

৭.১৪ Debugging নির্দেশিকা

Object-এর Bug ধরতে প্রথম কাজ: Object-টির আসল চেহারা দেখা।

debug-object.js · JavaScript — চলবে: Node.js

```
const student = { id: 'S1', name: 'Sara', contact: null };

console.log(Object.keys(student));
console.log(JSON.stringify(student, null, 2));
console.log(Object.hasOwn(student, 'marks'));
```

Output

```
[ 'id', 'name', 'contact' ]
{
  "id": "S1",
  "name": "Sara",
  "contact": null
}
false
```

`Object.keys()` দেখায় আসলে কোন Property আছে; বানান ভুল (`marks` বনাম `mark`) সঙ্গে সঙ্গে ধরা পড়ে। `JSON.stringify(object, null, 2)` ভেতরের স্তরসহ পুরো Object সাজিয়ে দেখায়।

Object নিয়ে কাজ প্রত্যাশামতো না হলে:

১. পুরো Object `JSON.stringify(object, null, 2)` দিয়ে দেখুন।

২. `undefined` এলে `Object.keys()` দিয়ে Property-র আসল নাম মিলিয়ে নিন।

৩. `Cannot read properties of undefined (reading 'x')` এলে বার্তার শেষের নামটির আগের ধাপটি দেখুন: সেটিই `undefined`।

৪. একটি Object অপ্রত্যাশিতভাবে বদলালে খুঁজুন আর কোন Variable তার Reference ধরে আছে, এবং Copy-টি Shallow ছিল কি না।

৫. `this` সংক্রান্ত সমস্যায় দেখুন Method-টি কীভাবে Call হয়েছে: ডটের বাঁ পাশে Object আছে কি না।

৬. JSON পার্স ব্যর্থ হলে String-টি নিজে দেখুন; বার্তার `position` বলে দেয় কোন অক্ষরে সমস্যা।

৭.১৫ পেশাদার অভ্যাস

- **আগে Data Model, পরে Code।** প্রতিটি Property-র নাম, Type এবং নিয়ম লিখে নিন।
- **একই ধরনের সব Object-এ একই Property।** ঐচ্ছিক তথ্য না থাকলে `null` রাখুন, Property বাদ দেবেন না।
- **হিসাব করা যায় এমন তথ্য সংরক্ষণ করবেন না।** Grade, মোট, বয়স—এগুলো দরকারের সময় হিসাব করুন।
- **প্রতিটি রেকর্ডের একটি অনন্য `id` রাখুন।**
- **Argument-এর Object বদলাবেন না;** `{ ...object, key: newValue }` দিয়ে নতুন Object ফেরত দিন।
- **Parameter-এ Destructuring** ব্যবহার করুন, যাতে Function কোন তথ্য চায় তা প্রথম লাইনেই বোঝা যায়।
- **Key-এর নাম camelCase-এ,** হাইফেন বা ফাঁকা জায়গা ছাড়া; তাহলে সব জায়গায় Dot Notation চলে।
- **বাইরে থেকে আসা Object বিশ্বাস করবেন না।** JSON-এ আসা তথ্য ব্যবহারের আগে যাচাই করুন।

৭.১৬ Project: Student Management System

শিক্ষার্থীদের তথ্য সামলানোর একটি Program লিখবেন: যোগ, নম্বর বদলানো, মোছা, খোঁজা এবং সারসংক্ষেপ। অনুচ্ছেদ ৭.১২-এর Data Model এর ভিত্তি।

Project-এর চাহিদা

- প্রতিটি শিক্ষার্থী অনুচ্ছেদ ৭.১২-এর Model অনুযায়ী একটি Object।
- নতুন শিক্ষার্থী যোগ করার আগে যাচাই: ID ফাঁকা নয় এবং আগে থেকে নেই, নাম কমপক্ষে ২ অক্ষর, নম্বর ০–১০০-এর পূর্ণসংখ্যা। সমস্যা থাকলে কারণসহ বার্তা।
- যোগ, নম্বর বদলানো ও মোছার কাজ মূল Array বা Object বদলাবে না।
- নাম বা ID দিয়ে খোঁজা যাবে, বড়-ছোট হাতের অক্ষর নির্বিশেষে।
- সারসংক্ষেপ: মোট শিক্ষার্থী, গড় নম্বর (এক দশমিক ঘরে), উত্তীর্ণের সংখ্যা এবং সর্বোচ্চ নম্বরধারী।
- ফোন নম্বর না থাকলে "নেই" দেখাবে।

এখানে ব্যবহৃত ফোন নম্বরগুলো কাল্পনিক।

File-এর কাঠামো

Folder-এর কাঠামো

```
student-management/
├── students.js
├── README.md
└── tests/
    ├── load-script.js
    └── students.test.js
```

students.js

File-টি তিন অংশে দেখানো হলো; তিনটি মিলে সম্পূর্ণ File। অংশগুলো পরপর একই File-এ লিখুন। `// ...` লাইনগুলো শুধু অংশের সংযোগ বোঝায়; সেগুলো লিখতে হবে না।

অংশ ১: Grade এবং যাচাই

students.js · JavaScript অংশ ১ / ৩ (Part 1 of 3) চলবে: Node.js

```
'use strict';

const PASS_MARK = 33;
const GRADES = [
  { min: 80, grade: 'A+' },
  { min: 70, grade: 'A' },
  { min: 60, grade: 'A-' },
  { min: 50, grade: 'B' },
```

```
  { min: 40, grade: 'C' },
  { min: 33, grade: 'D' },
  { min: 0, grade: 'F' },
];

// নম্বর থেকে Grade।
function gradeFor(marks) {
  const { grade } = GRADES.find((row) => marks >= row.min);
  return grade;
}

// নতুন শিক্ষার্থীর তথ্য যাচাই করে; সমস্যা থাকলে বার্তা, না থাকলে null।
function validateStudent(input, students) {
  const id = String(input.id ?? '').trim();
  const name = String(input.name ?? '').trim();
  const marks = input.marks;

  if (id === '') {
    return 'ID দিতে হবে।';
  }

  if (name.length < 2) {
    return 'নাম কমপক্ষে ২ অক্ষরের হতে হবে।';
  }

  if (!Number.isInteger(marks) || marks < 0 || marks > 100) {
    return 'নম্বর ০ থেকে ১০০-এর মধ্যে পূর্ণসংখ্যা হতে হবে।';
  }

  if (students.find((student) => student.id === id) !== undefined) {
    return `ID ${id} আগে থেকেই আছে।`;
  }

  return null;
}

// ...
```

Grade-এর সীমাগুলো Code-এর মধ্যে `if` হিসেবে না লিখে একটি Object-এর Array-তে রাখা হয়েছে। এটি তথ্য, তাই তথ্যের মতো করেই রাখা ভালো: সীমা বদলাতে শুধু টেবিলটি বদলালেই চলে। `GRADES` বড় থেকে ছোট ক্রমে সাজানো, তাই `find` প্রথম যে সারিতে নম্বর সীমা ছোঁয়, সেটিই দেয়; শেষ সারির সীমা `0`, তাই কোনো বৈধ নম্বরে `find` কখনো `undefined` দেয় না। তারপর Destructuring দিয়ে শুধু `grade` নেওয়া হয়েছে।

`validateStudent`-এ `String(input.id ?? '')` একসঙ্গে দুটি সমস্যা সামলায়: `id` না থাকলে ফাঁকা String, আর সংখ্যা হিসেবে এলে String-এ রূপান্তর।

অংশ ২: তালিকা বদলানো ও খোঁজা

students.js · JavaScript অংশ ২ / ৩ (Part 2 of 3) চলবে: Node.js

```
// ...

// যাচাই করে যোগ করে। ফেরত দেয় { students, error }।
function addStudent(students, input) {
  const error = validateStudent(input, students);

  if (error !== null) {
    return { students, error };
  }

  const student = {
    id: input.id.trim(),
    name: input.name.trim(),
    marks: input.marks,
    contact: input.contact ?? null,
  };

  return { students: [...students, student], error: null };
}

// একজনের নম্বর বদলে নতুন Array; বাকিরা অপরিবর্তিত।
function updateMarks(students, id, marks) {
  return students.map((student) =>
    student.id === id ? { ...student, marks } : student,
  );
}

// নির্দিষ্ট ID বাদ দিয়ে নতুন Array।
const removeStudent = (students, id) =>
  students.filter((student) => student.id !== id);

// নাম বা ID-তে খোঁজে; বড়-ছোট হাতের অক্ষর নির্বিশেষে।
function searchStudents(students, query) {
  const text = query.trim().toLowerCase();

  return students.filter(({ id, name }) =>
    id.toLowerCase().includes(text) || name.toLowerCase().includes(text),
  );
}

// ...
```

`addStudent` দুটি তথ্য ফেরত দিতে চায়: নতুন তালিকা এবং সম্ভাব্য সমস্যা। একটি Function একটিই মান ফেরত দিতে পারে, তাই দুটি একটি Object-এ রাখা হয়েছে; Shorthand Property-র কারণে `{ students, error }` লেখা গেছে। সমস্যা থাকলে মূল তালিকাটিই অপরিবর্তিত ফেরত যায়।

`updateMarks`-এ `{ ...student, marks }` হলো অনুচ্ছেদ ৭.৯-এর Spread এবং অনুচ্ছেদ ৭.২-এর Shorthand একসঙ্গে: পুরোনো সব তথ্য, শুধু `marks` নতুন। `searchStudents`-এর Callback Parameter-এ Destructuring করে শুধু `id` ও `name` নিয়েছে। `toLowerCase()` সব অক্ষর ছোট হাতের করে, আর String-এর `includes()` জানায় একটি লেখা আরেকটির ভেতরে আছে কি না; দুটিই অধ্যায় ৮-এ বিস্তারিত আসবে।

অংশ ৩: সারসংক্ষেপ, দেখানো এবং উদাহরণ

students.js · JavaScript অংশ ৩ / ৩ (Part 3 of 3) চলবে: Node.js

```
// ...

// মোট, গড়, উত্তীর্ণ এবং সর্বোচ্চ নম্বরধারীর নাম।
function summarize(students) {
  const total = students.length;

  if (total === 0) {
    return { total, average: 0, passed: 0, topper: null };
  }

  const sum = students.reduce((acc, { marks }) => acc + marks, 0);
  const passed = students.filter(({ marks }) => marks >= PASS_MARK).length;
  const best = students.reduce(
    (top, student) => (student.marks > top.marks ? student : top),
    students[0],
  );
  const average = Math.round((sum / total) * 10) / 10;

  return { total, average, passed, topper: best.name };
}

// একজন শিক্ষার্থীর এক লাইনের বিবরণ।
function formatStudent({ id, name, marks, contact }) {
  const phone = contact?.phone ?? 'নেই';
  return `${id} ${name}: ${marks} (${gradeFor(marks)}), ফোন: ${phone}`;
}

let students = [];
const inputs = [
  {
    id: 'S1',
    name: 'Sara Ahmed',
    marks: 88,
    contact: { phone: '01700-000001' },
  },
  { id: 'S2', name: 'Rafi', marks: 28 },
  {
    id: 'S3',
    name: 'Mina Das',
    marks: 74,
```

```
    contact: { phone: '01700-000003' },
  },
  { id: 'S2', name: 'Another Rafi', marks: 50 },
  { id: 'S4', name: 'N', marks: 60 },
];

for (const input of inputs) {
  const result = addStudent(students, input);
  students = result.students;

  if (result.error !== null) {
    console.log(`বাতিল: ${result.error}`);
  }
}

students = updateMarks(students, 'S2', 35);

for (const student of students) {
  console.log(formatStudent(student));
}

const { total, average, passed, topper } = summarize(students);
console.log(`মোট ${total} জন, গড় ${average}, উত্তীর্ণ ${passed} জন, সর্বোচ্চ: ${topper}`);
console.log(searchStudents(students, 'MINA').map(({ name }) => name).join(', '));
```

Output

```
বাতিল: ID S2 আগে থেকেই আছে।
বাতিল: নাম কমপক্ষে ২ অক্ষরের হতে হবে।
S1 Sara Ahmed: 88 (A+), ফোন: 01700-000001
S2 Rafi: 35 (D), ফোন: নেই
S3 Mina Das: 74 (A), ফোন: 01700-000003
মোট 3 জন, গড় 65.7, উত্তীর্ণ 3 জন, সর্বোচ্চ: Sara Ahmed
Mina Das
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **যাচাই আগে, যোগ পরে।** পাঁচটি ইনপুটের মধ্যে দুটি বাতিল হয়েছে: একই ID দ্বিতীয়বার, এবং এক অক্ষরের নাম। প্রতিটির কারণ আলাদা বার্তায়।
- **`students = result.students`** — `students` একটি `let`, কারণ প্রতিবার নামটিকে নতুন Array-র দিকে সরানো হচ্ছে। কোনো Array নিজে বদলাচ্ছে না।
- **`updateMarks`-এর পর।** Rafi-র নম্বর ২৮ থেকে ৩৫ হয়েছে, তাই Grade `D` এবং তিনি এখন উত্তীর্ণ।
- **`summarize`-এ `reduce` দুবার।** প্রথমবার যোগফল; দ্বিতীয়বার সর্বোচ্চ নম্বরধারী, যেখানে শুরুর মান হিসেবে প্রথম শিক্ষার্থীকে দেওয়া হয়েছে (অধ্যায় ৬-এর নিয়ম)। গড় ১৯৭ ÷ ৩ = ৬৫.৬৬..., এক দশমিক ঘরে ৬৫.৭।

- **ফাঁকা তালিকা।** `summarize` শুরুতেই ফাঁকা তালিকা আলাদা করে সামলায়; নইলে শূন্য দিয়ে ভাগ হতো এবং `students[0]` হতো `undefined`।
- **`formatStudent`-এ Destructuring ও Optional Chaining।** Rafi-র `contact` হলো `null`, তাই `contact?.phone` `undefined` দেয় এবং `??` "নেই" বসায়।
- **শেষে Destructuring।** `summarize`-এর ফেরত দেওয়া Object থেকে চারটি মান একবারে আলাদা করা হয়েছে।
- **খোঁজা।** `'MINA'` বড় হাতের অক্ষরে দেওয়া হলেও Mina Das পাওয়া গেছে।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node students.js
```

অবস্থা	যা হওয়ার কথা
একই ID দ্বিতীয়বার	বাতিল: ID S2 আগে থেকেই আছে।
এক অক্ষরের নাম	বাতিল: নাম কমপক্ষে ২ অক্ষরের হতে হবে।
নম্বর `101`, `-1`, `88.5` বা `'88'`	বাতিল: নম্বর ০ থেকে ১০০-এর মধ্যে পূর্ণসংখ্যা হতে হবে।
ফোন নম্বর ছাড়া	ফোন: নেই
`updateMarks`	শুধু সেই শিক্ষার্থীর নম্বর বদলায়; মূল তালিকা অক্ষত
ফাঁকা তালিকার সারসংক্ষেপ	মোট 0, গড় 0, উত্তীর্ণ 0, সর্বোচ্চ `null`
খোঁজা `'mina'` বা `'MINA'`	Mina Das

Companion Source Code-এ এই Project-এর ১৬টি Automated Test আছে। চালানোর Command `README.md`-তে আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
একই ID দুবার যোগ হয়ে যায়	`validateStudent`-এ পুরোনো তালিকা পাঠানো হয়নি	`validateStudent(input, students)`
নম্বর বদলানোর পর মূল তালিকাও বদলে যায়	`student.marks = marks` লেখা হয়েছে	`{ ...student, marks }`
`TypeError: Cannot read properties of null (reading 'phone')`	`contact.phone` লেখা হয়েছে	`contact?.phone`

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Grade সব সময় `F`	`GRADES` ছোট থেকে বড় ক্রমে সাজানো	বড় থেকে ছোট ক্রমে রাখুন
`TypeError: Cannot destructure property 'grade' of 'undefined'`	কোনো সীমা মেলেনি (যেমন ঋণাত্মক নম্বর)	যাচাই আগে করুন; শেষ সারির সীমা `0` রাখুন
গড় `NaN`	ফাঁকা তালিকার যাচাই নেই	`total === 0` আগে যাচাই

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। তথ্য Program-এর ভেতরে লেখা এবং কোথাও সংরক্ষিত হয় না; অনুশীলনী ১২-তে JSON দিয়ে সংরক্ষণের প্রথম ধাপ আছে। ফোন নম্বরের গঠন যাচাই করা হয় না। নাম ও ID-র খোঁজা খুব সরল; বাংলা নামের বানানভেদ সামলানো হয় না (অধ্যায় ৮ দেখুন)। নম্বর বদলানোর সময় নতুন নম্বর যাচাই করা হয় না।

৭.১৭ অনুশীলনী

প্রাথমিক স্তর

১. একটি `book` Object তৈরি করুন, যাতে `title`, `author` ও `price` আছে। তারপর Spread দিয়ে দাম ১০ শতাংশ কমিয়ে একটি নতুন Object তৈরি করুন; মূল Object যেন না বদলায়।

২. `getFullName` নামে একটি Function লিখুন, যা `{ firstName, lastName }` আকারের Object নিয়ে পুরো নাম ফেরত দেয়। Parameter-এ Destructuring ব্যবহার করুন।

৩. `{ bangla: 80, english: 72, math: 91 }` থেকে `Object.entries()` দিয়ে প্রতিটি বিষয় ও নম্বর দেখান।

৪. না চালিয়ে বলুন: `const user = { profile: null }; user.profile?.email ?? 'নেই'`-এর মান কী, এবং কেন।

৫. নিচের Code-এ `price` দেখানোর কথা, কিন্তু দেখাচ্ছে `undefined`। ভুলটি ঠিক করুন।

key-bug.js · JavaScript — চলবে: Browser Console ও Node.js

```
const product = { name: 'Pen', price: 15 };
const field = 'price';

console.log(product.field);
```

মধ্যম স্তর

৬. `Object.hasOwn()` এবং `in`-এর পার্থক্য একটি উদাহরণ দিয়ে দেখান।

৭. শিক্ষার্থীদের একটি Array থেকে যাঁদের নম্বর ৮০ বা বেশি, তাঁদের নাম `filter`, `map` এবং Destructuring দিয়ে বের করুন।

৮. `counter` নামে একটি Object লিখুন, যাতে `count` Property এবং `increment()` ও `reset()` Method আছে। `this` ব্যবহার করুন।

৯. `moveToCity` নামে একটি Function লিখুন, যা শিক্ষার্থীর Object এবং একটি শহরের নাম নিয়ে নতুন Object ফেরত দেয়, যেখানে `contact.city` বদলেছে, কিন্তু মূল Object এবং তার `contact` অপরিবর্তিত।

চ্যালেঞ্জ স্তর

১০. **বিষয়ভিত্তিক গড়:** কয়েকজন শিক্ষার্থীর বিষয়ভিত্তিক নম্বর (`{ name, marks: { bangla, english } }`) থেকে প্রতিটি বিষয়ের গড় বের করুন।

১১. **সমতা যাচাই:** `sameStudent` নামে একটি Function লিখুন, যা দুটি শিক্ষার্থী-Object-এর `id`, `name` ও `marks` মিলিয়ে বিষয়বস্তু একই কি না বলে।

১২. **সংরক্ষণ ও পুনরুদ্ধার:** শিক্ষার্থীদের একটি Array JSON String-এ রূপান্তর করুন, তারপর সেটি থেকে আবার Array তৈরি করুন এবং দেখান যে দুটির বিষয়বস্তু একই, কিন্তু তারা আলাদা Array।

৭.১৮ অনুশীলনীর সমাধান

সমাধান ১

book.js · JavaScript — চলবে: Browser Console ও Node.js

```
const book = { title: 'JavaScript', author: 'Moynul', price: 500 };
const discounted = { ...book, price: book.price - book.price * 10 / 100 };

console.log(discounted.price);
console.log(book.price);
```

Output

```
450
500
```

সমাধান ২

full-name.js · JavaScript চলবে: Browser Console ও Node.js

```
function getFullName({ firstName, lastName }) {
  return `${firstName} ${lastName}`;
}

console.log(getFullName({ firstName: 'Sadia', lastName: 'Rahman' }));
```

Output

```
Sadia Rahman
```

সমাধান ৩

subjects.js · JavaScript চলবে: Browser Console ও Node.js

```
const marks = { bangla: 80, english: 72, math: 91 };

for (const [subject, mark] of Object.entries(marks)) {
  console.log(`${subject}: ${mark}`);
}
```

Output

```
bangla: 80
english: 72
math: 91
```

সমাধান ৪ — `'নেই'`। `user.profile` হলো `null`, তাই `?.` সেখানেই থেমে `undefined` দেয়; তারপর `??` বিকল্প মান `'নেই'` বসায়। `?.` না থাকলে `TypeError` হতো।

সমাধান ৫ — `product.field` Variable-এর মান ব্যবহার করে না; `'field'` নামের Property খোঁজে। Variable-এর মান দিয়ে খুঁজতে বর্গবন্ধনী লাগে।

key-fixed.js · JavaScript চলবে: Browser Console ও Node.js

```
const product = { name: 'Pen', price: 15 };
const field = 'price';

console.log(product[field]);
```

Output

```
15
```

সমাধান ৬

has-own-vs-in.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const item = { name: 'Pen' };

console.log('name' in item, Object.hasOwn(item, 'name'));
console.log('toString' in item, Object.hasOwn(item, 'toString'));
```

Output

```
true true
true false
```

`toString` Object-টির নিজের Property নয়, উত্তরাধিকারসূত্রে পাওয়া। `in` সেটিও গোনে; `Object.hasOwn()` গোনে না।

সমাধান ৭

top-students.js · JavaScript — চলবে: Browser Console ও Node.js

```js
const students = [
  { name: 'Sara', marks: 88 },
  { name: 'Rafi', marks: 35 },
  { name: 'Mina', marks: 80 },
];

const toppers = students
  .filter(({ marks }) => marks >= 80)
  .map(({ name }) => name);

console.log(toppers.join(', '));
```

Output

```
Sara, Mina
```

সমাধান ৮

counter.js · JavaScript — চলবে: Browser Console ও Node.js

```
const counter = {
  count: 0,

  increment() {
    this.count++;
    return this.count;
  },

  reset() {
    this.count = 0;
  },
};

counter.increment();
console.log(counter.increment());
counter.reset();
console.log(counter.count);
```

Output

```
2
0
```

সমাধান ৯

move-city.js · JavaScript — চলবে: Browser Console ও Node.js

```
function moveToCity(student, city) {
  return { ...student, contact: { ...student.contact, city } };
}

const student = {
  name: 'Mina',
  contact: { phone: '01700-000003', city: 'Khulna' },
};
const moved = moveToCity(student, 'Sylhet');

console.log(moved.contact.city, moved.contact.phone);
console.log(student.contact.city);
```

Output

```
Sylhet 01700-000003
Khulna
```

যে স্তরটি বদলাচ্ছে, শুধু সেই স্তরে আবার Spread করা হয়েছে; ফোন নম্বর নতুন Object-এও আছে, আর মূল `contact` অক্ষত।

সমাধান ১০

subject-average.js · JavaScript চলবে: Browser Console ও Node.js

```
const students = [
  { name: 'Sara', marks: { bangla: 80, english: 90 } },
  { name: 'Rafi', marks: { bangla: 60, english: 70 } },
];

const totals = {};

for (const { marks } of students) {
  for (const [subject, mark] of Object.entries(marks)) {
    totals[subject] = (totals[subject] ?? 0) + mark;
  }
}

for (const [subject, sum] of Object.entries(totals)) {
  console.log(`${subject}: ${sum / students.length}`);
}
```

Output

```
bangla: 70
english: 80
```

`totals[subject]`-এ বর্গবন্ধনী লাগছে, কারণ Key-টি Variable-এ আছে। প্রথমবার Key-টি থাকে না, তাই `?? 0` দিয়ে শূন্য থেকে শুরু করা হয়েছে। এখানে `totals` Function-এর বাইরে নয়, নিজের তৈরি একটি স্থানীয় Object, তাই তাতে সরাসরি মান যোগ করা নিরাপদ।

সমাধান ১১

same-student.js · JavaScript চলবে: Browser Console ও Node.js

```
function sameStudent(a, b) {
  return a.id === b.id && a.name === b.name && a.marks === b.marks;
}

const first = { id: 'S1', name: 'Sara', marks: 88 };
const copy = { ...first };
const changed = { ...first, marks: 90 };

console.log(first === copy);
console.log(sameStudent(first, copy));
console.log(sameStudent(first, changed));
```

Output

```
false
true
false
```

সমাধান ১২

save-restore.js · JavaScript চলবে: Browser Console ও Node.js

```
const students = [
  { id: 'S1', name: 'Sara', marks: 88 },
  { id: 'S2', name: 'Rafi', marks: 35 },
];

const saved = JSON.stringify(students);
const restored = JSON.parse(saved);

console.log(restored.length, restored[1].name);
console.log(JSON.stringify(restored) === saved);
console.log(restored === students);
```

Output

```
2 Rafi
true
false
```

বাস্তবে `saved` String-টি একটি File বা `localStorage`-এ রাখা হতো। পুনরুদ্ধারের পর তথ্য যাচাই করা উচিত, কারণ সংরক্ষিত লেখাটি বদলে গিয়ে থাকতে পারে।

৭.১৯ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `const key = 'name';` থাকলে `user.name`-এর সমান কোনটি?

ক) `user.key`

খ) `user['key']`

গ) `user[key]`

ঘ) `user.[key]`

প্রশ্ন ২. `const name = 'Rafi'; const s = { name };` লেখার পর `s` কী?

ক) `{ name: 'Rafi' }`

খ) `{ 'Rafi': name }`

গ) একটি Error

ঘ) `{ name: undefined }`

প্রশ্ন ৩. একটি Property আছে কিন্তু তার মান `undefined`। কোনটি নির্ভরযোগ্যভাবে বলে Property-টি আছে?

ক) `obj.key === undefined`

খ) `Object.hasOwn(obj, 'key')`

গ) `obj.key !== null`

ঘ) `typeof obj.key`

প্রশ্ন ৪. `account.deposit(100)` Call-এ `deposit`-এর ভেতরে `this` কী?

ক) `deposit` Function

খ) Global Object

গ) `undefined`

ঘ) `account`

প্রশ্ন ৫. `const a = { contact: null }; a.contact?.phone`-এর মান কী?

ক) `undefined`

খ) `null`

গ) একটি `TypeError`

ঘ) ফাঁকা String

প্রশ্ন ৬. `const { marks: score } = { marks: 88 };` লেখার পর কোন Variable তৈরি হয়?

ক) `marks`, মান `88`

খ) `score`, মান `88`

গ) দুটোই

ঘ) কোনোটিই নয়; এটি একটি Error

প্রশ্ন ৭. `const copy = { ...original };`-এর পর `copy.contact.city` বদলালে কী হয় (`contact` একটি Object)?

ক) শুধু `copy` বদলায়

খ) একটি Error হয়

গ) `original.contact.city`-ও বদলায়

ঘ) কিছুই বদলায় না

প্রশ্ন ৮. `Object.entries({ a: 1, b: 2 })` কী ফেরত দেয়?

ক) `[ [ 'a', 1 ], [ 'b', 2 ] ]`

খ) `[ 'a', 'b' ]`

গ) `[ 1, 2 ]`

ঘ) `{ a: 1, b: 2 }`

প্রশ্ন ৯. `JSON.stringify({ a: undefined, b: 2 })`-এর ফল কী?

ক) `{"a":undefined,"b":2}`

খ) `{"a":null,"b":2}`

গ) একটি Error

ঘ) `{"b":2}`

প্রশ্ন ১০. `{ id: 1 } === { id: 1 }`-এর ফল কী?

ক) `true`, কারণ বিষয়বস্তু একই

খ) `false`, কারণ এরা দুটি আলাদা Object

গ) একটি Error

ঘ) `undefined`

৭.২০ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–খ · ৪–ঘ · ৫–ক · ৬–খ · ৭–গ · ৮–ক · ৯–ঘ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	বর্গবন্ধনী Variable-এর মান ব্যবহার করে। `user.key` খোঁজে `'key'` নামের Property।
২	ক	Shorthand Property: Key এবং Variable-এর নাম এক হলে একবার লিখলেই চলে।
৩	খ	`=== undefined` "নেই" এবং "আছে কিন্তু `undefined`"-এর পার্থক্য করতে পারে না।
৪	ঘ	Call-এর সময় ডটের বাঁ পাশের Object-ই `this`।
৫	ক	`?.` বাঁ পাশ `null` হলে থেমে `undefined` দেয়; Error হয় না।
৬	খ	কোলনের বাঁ পাশে Property-র নাম, ডান পাশে নতুন Variable-এর নাম।
৭	গ	Spread Copy Shallow; ভেতরের Object-এর শুধু ঠিকানা Copy হয়।
৮	ক	প্রতিটি Property একটি `[key, value]` জোড়া হয়ে আসে।
৯	ঘ	JSON-এ `undefined` রাখা যায় না, তাই Property-টি বাদ পড়ে।
১০	খ	`===` Object-এর ঠিকানা তুলনা করে, বিষয়বস্তু নয়।

৭.২১ অধ্যায়ের সারসংক্ষেপ

- Object তথ্য রাখে নাম (Key) অনুযায়ী; Array রাখে অবস্থান অনুযায়ী।
- Shorthand Property (`{ name }`) এবং Computed Key (`{ [field]: value }`) Object লেখা সংক্ষিপ্ত করে।
- Key জানা থাকলে Dot; Key Variable-এ থাকলে বা বিশেষ অক্ষর থাকলে Bracket।
- Property যোগ ও বদল একইভাবে লেখা হয়; `delete` সরায়; অস্তিত্ব যাচাইয়ে `Object.hasOwn()`।
- Method হলো Function-মানের Property; ভেতরে `this` হলো যে Object দিয়ে Call করা হয়েছে। Method Arrow Function দিয়ে লিখবেন না।
- `?.` ঐচ্ছিক Nested তথ্য নিরাপদে পড়ে; `??` বিকল্প মান দেয়।
- Destructuring Object ও Array থেকে মান আলাদা Variable-এ নেয়; Parameter-এও ব্যবহার করা যায়।
- `Object.keys`, `Object.values`, `Object.entries` Object থেকে Array তৈরি করে।
- Spread Copy Shallow; সম্পূর্ণ আলাদা Copy-র জন্য `structuredClone()`।
- `===` ঠিকানা তুলনা করে; `Object.freeze()`-ও Shallow।
- JSON হলো তথ্যের String-রূপ: `stringify` এবং `parse`। `undefined`, Function বাদ পড়ে; তারিখ String হয়ে যায়।
- আগে Data Model, পরে Code।

পরের অধ্যায়ে String এবং লেখা প্রক্রিয়াকরণ: খোঁজা, বদলানো, ভাগ করা, Unicode এবং বাংলা লেখার বিশেষ সমস্যা।

৭.২২ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Property	Object-এর একটি নাম-মানের জোড়া।
Key / Value	Property-র নাম / তার মান।
Object Literal	`{ }` দিয়ে লেখা Object।
Shorthand Property	Key ও Variable একই নামে হলে একবার লেখা।
Computed Key	`[ ]` -এর Expression থেকে তৈরি Key।
Method	Function-মানের Property।
`this`	Method Call-এর সময় যে Object দিয়ে Call করা হয়েছে।
Optional Chaining	`?.` : মান না থাকলে থেমে `undefined` দেওয়া।
Destructuring	Object বা Array থেকে মান আলাদা Variable-এ নেওয়া।
Shallow Copy	শুধু প্রথম স্তর Copy; ভেতরের Object-এর ঠিকানা ভাগাভাগি হয়।
Deep Copy	সব স্তরসহ সম্পূর্ণ আলাদা Copy।
JSON	তথ্যকে String হিসেবে প্রকাশের মান-নির্ধারিত রূপ।
Data Model	একটি জিনিসের তথ্যের গঠন ও নিয়মের নকশা।

৭.২৩ আরও পড়ুন

- **MDN — Working with objects:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_objects` — Object-এর পূর্ণ নির্দেশিকা।
- **MDN — Destructuring:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring` — Destructuring-এর সব রূপ।
- **MDN — Optional chaining:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Optional_chaining` — `?.` -এর নিয়ম।
- **MDN — this:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this` — `this` -এর আচরণ।
- **MDN — JSON:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON` — `JSON.stringify` ও `JSON.parse`।